

Compiler Construction CA2: Semantic Analysis and Intermediate Representation

Disclaimer:

Appendix I: Student Declaration of Academic Integrity

Students may be required to submit work for assessment in a variety of means, for example physical submission or electronic submission as per the lecturer's instructions. In all cases students must make a declaration of academic integrity, either by physically completing such a declaration and submitting it with their assignment or engaging appropriately with the electronic version of the declaration. Assignments submitted such that the form has not been included, or the electronic equivalent has been circumvented, will not be accepted.

Declaration

NAME: Ciaran Farrell

STUDENT ID NUMBER

PROGRAMME 22350403

MODULE CODE CSC1098

ASSIGNMENT TITLE Assignment 2: Semantic Analysis and Intermediate Representation

SUBMISSION DATE 01/12/2025

I understand that the University regards breaches of academic integrity and plagiarism as grave and serious.

I have read and understood the DCU Academic Integrity and Plagiarism Policy. I accept the penalties that may be imposed should I engage in practice or practices that breach this policy.

I have identified and included the source of all facts, ideas, opinions and viewpoints of others in the assignment references. Direct quotations, paraphrasing, discussion of ideas from books, journal articles, internet sources, module text, or any other source whatsoever are acknowledged and the sources cited are identified in the assignment references.

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others save and to the extent that such work has been cited and acknowledged within the text of my work.

I have used the DCU library referencing guidelines (available at https://www4.dcu.ie/library/classes_and_tutorials/citingreferencing.shtml)

and/or the appropriate referencing system recommended in the assignment guidelines and/or programme documentation.

By signing this form or by submitting material online I confirm that this assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

By signing this form or by submitting material for assessment online I confirm that I have read and understood the DCU Academic Integrity and Plagiarism Policy (available at <http://www.dcu.ie/registry/examinations/index.shtml>).

Signature
Ciaran Farrell

Date:
30/11/25

Main Pipeline.....	4
CCL Parser/Scanner Extensions.....	4
CCLType.java.....	4
Abstract Syntax Tree.....	4
AST.java.....	4
Base Class.....	4
Visitor.....	4
Specific Abstract Classes (Nodes).....	5
ExpressionNode.....	5
DeclNode.....	5
StatementNode.....	5
Node Classes (Non-Abstract).....	5
ASTBuilder.java.....	6
Symbol Table.....	6
Helpers.....	6
SymbolKind.java.....	6
SymbolEntry.java.....	6
Scope.java.....	6
SymbolTable.java.....	7
Semantic Checking.....	7
SemanticAnalyser.java.....	7
Intermediate Code Generation.....	8
IRGenerator.java.....	8
Example Code Generation.....	10
Test A:.....	10
Test B:.....	12
References/Bibliography.....	15

Files for completion of assignment

- AST.java
- ASTBuilder.java
- CCL.g4
 - With generated files
- CCLType.java
- IRGenerator.java
- Main.java
- Scope.java
- SemanticAnalyser.java
- SymbolEntry.java
- SymbolKind.java
- SymbolTable.java
- Extra:
 - TACi.jar

Main Pipeline

Each of the following files are all ran by a main file which is outlined in the following:

- Check for arguments
- Check for .ccl filetype
- Change .ccl to .ir
- Parse through input file
 - Standard parsing with small changes from assignment 1, mainly throwing specific lexer and syntax errors
- Semantic analyser ran and checked for specific error checks within the semantic analyser
- Generate the IR code with a message confirming it (as well as messages throughout confirming each stage of the process)

CCL Parser/Scanner Extensions

No major changes or issues found with original CCL.g4 grammar from assignment 1 however extensions to the generated g4 code were made.

CCLType.java

A enum and type converter was used to convert string cases of the values Integer, Boolean and Void into type values. This would be useful for checking types within symbol table implementation.

Abstract Syntax Tree

AST.java

This file utilises a visitor pattern to allow for simple operations specifically within semantic analysis and code generation. It contains syntactic elements of the CCL programs once parsing is complete.

Base Class

First, an abstract ASTNode class is created to allow each node contain a location(line) as well as have operations occur whilst keeping node classes (not modifying them)

Visitor

The visitor interface allows operations on the different node types of the program, the semantic analyser and IR Generator will use this to visit each node and perform the respective operations needed.

Specific Abstract Classes (Nodes)

Specific nodes will have abstract classes as they can be used in regular Node classes repeatedly. They are described as follows:

ExpressionNode

This structures how expressions are done, using CCLType to grab the type of expression. This can be used in other node classes without duplicating code

DeclNode

Each declaration has a name and a type, so its name is set and so is its expression. There are specific Var and Const declarations used so having an abstract class be extended with them is helpful for keeping code clean

StatementNode

As there are many different types of statements (Comparison, function call etc), this abstract class is set to empty.

Node Classes (Non-Abstract)

While each specific node can be discussed in detail, the majority of nodes follow a similar format

- The program node contains the structure of the program grammar. The main function also represents its grammar block
- Function nodes contain the function definition with signature, body, variables and returns.
- Parameters store the individual function parameters
- Assignments are for variable assignment
- The function call statement just calls statements and wraps them
 - If and while follows their intended purpose (if condition then else then, while condition then)
- Expression node extensions will contain some value, operator or name depending on which expression is used.
- For nodes which have left and right usage, an expression node for left and right are added.
- For nodes with bools or ints, the nodes are set respectively.
- Statement nodes similarly contain names as well as conditions. Skip will be accepted as soon as class called

In general, each node follows its intended grammar format

ASTBuilder.java

This file acts sort of like a bridge between Antlr4s parse tree and the visitor ast above. This extends the base visitor of CCL.g4 and returns the visit methods of AST nodes.

Throughout creating and testing, debug messages have been used as specifically function, expression and statement visits were a difficulty during development. Fallback debug messages were also added, for example printing if a statement is null/empty.

Each visit follows a similar pattern: checking for precedence of keywords and then building the structure for that keyword.

For example: For the "IF" check in statement method

- Check to see if the keyword "IF" is present
- If it is then it builds a complete if then else structure processed in order

Utility classes also: Added visitTerminal to reduce clutter from Antlr (if not, Antlr creates AST nodes for every single token, harder to manage and debug), and aggregateResult which controls how child nodes results are combined when visiting parent (last non-null child wins).

Symbol Table

Helpers

SymbolKind.java

Symbol kind is a simple enum which classifies the symbols in the grammar (VARIABLE, CONSTANT, FUNCTION, PARAMETER). This is used in SemanticAnalysis for built in functions as well as symbol checking.

SymbolEntry.java

This has the entries for each symbol, with the parameters of name, type, kind. Included in this is line (mainly for debugging purposes), used (for checking and tracking), initialised (if variable has value) and isGlobal (local or global scope). Type is using a type check set from CCLType.

Scope.java

Checks if an entry is global or local, prevents duplicate names in the same scope.

SymbolTable.java

This is a stack approach to how scope is managed, doing last in first out checking for local and then checking global scope afterwards.

enterScope is the first function to be called and creates a new local scope by pushing a new false globalScope on the stack. Then, exitScope can be used to leave the current scope, checking if local scopes are not empty and popping the localScopes from the stack (so if for example: enter->exit->exit. Error would be thrown as the stack is empty. This is why the check is in place). addSymbol adds the symbol to the scope specified by checking its current scope. First if the local scopes is empty that means it's in the global scope, and if it has entries in local. The scope is then selected by peeking the top of the stack. Then the symbol is registered and returned.

The lookup function is called to lookup the full scope and is used specifically for variable resolution, whilst the lookupCurrentScope checks specifically just the current scope and is used for detecting duplicate entries.

Memory is managed for the symbol table as scopes are garbage collected when popped. Leaks in memory therefore don't arise when scope data is abandoned.

Semantic Checking

Previous two sections on AST and SymbolTable all come together for semantic analysis on the ccl code. AST nodes are created which are used by the semantic analysis to build a symbol table, which then the symbol table is used to check scope and resolve symbols.

SemanticAnalyser.java

SemanticAnalyser goes through each node of the AST and performs a number of checks to ensure that errors are caught before code generation.

The specific states of the symbol are managed and are analysed through the analyse function entry point. Built-in functions are added from the TACi spec also.

Then checks performed as follows:

- Variables/Constants cannot be declared using the void type
- Duplicate variables cannot exist
- Constant values must match the types they are declared for

- Duplicate functions cannot exist
- Return types must match the functions type
- Non-void functions must return a value
- Parameters cannot be declared as a void type
- Duplicate parameters cannot exist
- Variables in statements must be declared
- Statements cannot be assigned constant
- Types in statements cannot mismatch (e.g bool something int)
- Comparison operands must work with integers
- Equality operands need to be the same type
- Logical negation needs to be a boolean
- Arithmetic negation needs to be an integer
- Functions must exist
- Functions need to be of function type
- If condition must be boolean
- While condition must be boolean
- Logical OR and AND must be boolean
- Variables/Functions which are unused are flagged

While more semantic checks could be added, the ones which have been implemented I believe to be appropriate enough. Two checks which were omitted due to time and testing constraints were within function signature checking, specifically argument count checking and type matching. For this assignment, these functions were only validated based on if the arguments are expressions.

Intermediate Code Generation

Intermediate code generation will convert the AST created into Three-address code for TACi

IRGenerator.java

Firstly, states are managed:

- An list keeps track of the generated three address code
- Functions are tracked as well as labels and temporaries

Entry point contains defaults such as defaulting t and L, adding a header to the code, generating functions and then generating the main function ending in call `_exit, 0`.

Each Node is visited and checked where code is added to the list, which is then written to the `.ir` file.

Notes for each node visited is as follows:

- Main

- Variables are initialised and for each one checked if it is a Var or Const declaration, which then adds appropriate code for it.
- The main statements are then generated
- Function
 - Function label added
 - Parameters are processed and then code added in (getparam used)
 - Local variables are declared
 - Function body is generated and then return is given
- Expression Generation
 - Two expression classes are used, generateExpressions and generateSimpleExpressions
 - This was used to optimise non temp values
 - generateExpression
 - Adding identifiers
 - Numbers for temp values
 - Boolean values
 - Arithmetic values
 - Comparison operators
 - All of these with the specific three address code code format
 - Simple expressions used to catch simple expressions, which if no checks are made will fallback to normal generated expressions.
- Generate Function call
 - In reverse order, push each parameter (as in TACi call convention)
 - Function is then called and input as in spec
- Generate Conditions
 - Handled as follows:
 - For logical statements (OR/AND)
 - OR: left true? Goto next, else check right
 - AND: left false? Goto false, else check right
 - If comparison statements:
 - Add specific TAC code ifz x relop y goto L
 - The equality check is similar but adding "==" and "!=" instead
 - Logical negation creates a temporary label and then creates a condition.
- While statements
 - Creates a new start and end label for the condition with the body in between

Note, the .ir file when generated is formatted for ease of reading as follows:

```
L1:
ifz x > 5 goto L3
output = 50
```

```
goto L4
```

Note that spaces are used as indentation.

Certain nodes are empty for the following reasons:

- The ASTVisitor interface requires these visitor methods to be present
- Specific nodes such as the declaration nodes and expression nodes are handled in the context of main/function and generate functions respectively.

Example Code Generation

Test A:

CCL

```
var result : integer;

integer conditional(x : integer)
{
    var output : integer;
    output = 0;

    if (x > 10)
    {
        output = 100;
    }
    else
    {
        if (x > 5)
        {
            output = 50;
        }
        else
        {
            output = 10;
        }
    }

    return (output);
}

main
{
    var arg : integer;
    arg = 10;
    result = conditional(arg);
    _println(result);
}
```

IR

// CCL File generated in 3-Addr Code for TACi

```
conditional:
x = getparam 1
output = 0
output = 0
ifz x > 10 goto L1
output = 100
```

```

    goto L2
L1:
    ifz x > 5 goto L3
    output = 50
    goto L4
L3:
    output = 10
L4:
L2:
    return output

main:
    arg = 0
    arg = 1
    param arg
    t1 = call conditional, 1
    result = t1
    param result
    call _println, 1
    call _exit, 0

```

Terminal

```

$ javac *.java
$ java Main test3.ccl
First Begin Parsing test3.ccl
Then Run SemanticAnalyser...
Next Generate IR Code...
Successfully Generated test3.ir
$ java -jar TACi.jar -d test3.ir
Three Address Code Interpreter (TACi) v1.1
TAC source test3.ir parsed successfully

```

AST Dump:

```

>Program
> Stms
>  Stm
>  Stms
>    Stm
>    Label
>    Identifier
...
>                                     Stms
>                                     Stm
>                                     PCall
>                                     Identifier
>                                     Number
>                                     Stms
>                                     Stm
>                                     PCall
>                                     Identifier
>                                     Number
>                                     Stms
10

```

Debug - Printing internal tables

```

Values:
output: 10
result: 10
arg: 1
x: 1
t1: 10

```

Entry Points:

```

conditional
L1
L2

```

L3
L4
main

Pointers:

Test B:

CCL

```
integer multiply (x : integer, y : integer)
{
    var result : integer;
    var minus_sign : boolean;

    if (x < 0 && y >= 0)
    {
        minus_sign = true;
        x = -x;
    }
    else
    {
        if (y < 0 && x >= 0)
        {
            minus_sign = true;
            y = -y;
        }
        else
        {
            if (x < 0 && y < 0)
            {
                minus_sign = false;
                x = -x;
                y = -y;
            }
            else
            {
                minus_sign = false;
            }
        }
    }

    result = 0;

    while (y > 0)
    {
        result = result + x;
        y = y - 1;
    }
    if (minus_sign != false)
    {
        result = -result;
    }
    else
    {
        skip;
    }

    return (result);
}

main
{
    var arg_1 : integer;
    var arg_2 : integer;
    var result : integer;
```

```

    arg_1 = 100;
    arg_2 = 100;

    _println(arg_1);
    _println(arg_2);
    // a simple comment
    /* a comment */ with /* several */ nested /* comments */

    result = multiply (arg_1, arg_2);

    _println(result);
}

```

IR

// CCL File generated in 3-Addr Code for TACi

```

multiply:
    x = getparam 1
    y = getparam 2
    result = 0
    minus_sign = 0
    ifz x < 0 goto L1
L3:
    ifz y >= 0 goto L1
    minus_sign = 1
    t1 = 0 - x
    x = t1
    goto L2
L1:
    ifz y < 0 goto L4
L6:
    ifz x >= 0 goto L4
    minus_sign = 1
    t2 = 0 - y
    y = t2
    goto L5
L4:
    ifz x < 0 goto L7
L9:
    ifz y < 0 goto L7
    minus_sign = 0
    t3 = 0 - x
    x = t3
    t4 = 0 - y
    y = t4
    goto L8
L7:
    minus_sign = 0
L8:
L5:
L2:
    result = 0
L10:
    ifz y > 0 goto L11
    t5 = result + x
    result = t5
    t6 = y - 1
    y = t6
    goto L10
L11:
    ifz minus_sign != 0 goto L12
    t7 = 0 - result
    result = t7
    goto L13
L12:
L13:

```

```
main:
    arg_1 = 0
    arg_2 = 0
    result = 0
    arg_1 = 100
    arg_2 = 100
    param arg_1
    call _println, 1
    param arg_2
    call _println, 1
    param arg_2
    param arg_1
    t8 = call multiply, 2
    result = t8
    param result
    call _println, 1
    call _exit, 0
```

```
$ java Main test4.ccl
First Begin Parsing test4.ccl
Then Run SemanticAnalyser...
Next Generate IR Code...
Successfully Generated test4.ir
$ java -jar TACi.jar -d test4.ir
Three Address Code Interpreter (TACi) v1.1
TAC source test4.ir parsed successfully
```

• • •

```

Stms
  Stm
    PCall
      Identifier
        Number
Stms
  Stm
    PCall

```

	Identifier
	Number
	Stms
>	
>	
>	
100	
100	
10000	

Debug - Printing internal tables

Values:

result: 10000
t5: 10000
minus_sign: 0
t6: 0
arg_1: 100
t8: 10000
arg_2: 100
x: 100
y: 0

Entry Points:

L1
L2
L3
L4
L5
main
L10
L6
L7
L12
L8
L11
L9
L13
multiply

Pointers:

References/Bibliography

Parr, T. (2013). Definitive ANTLR 4 Reference, 2nd Edition.

- Used as a general guide for ANTLR4, used throughout development as a reference for certain naiveties

Sinclair. D, "Semantic Analysis"

Sinclair. D, "Intermediate Representation"

- Both of these sets of slides were used as reference for their respective topic, specifically used for the information on Visitor pattern as well as 3-Addr Code

Antlr4 Documentation:

<https://github.com/antlr/antlr4/blob/master/doc/index.md>

Java Documentation: <https://docs.oracle.com/en/java/>

- (mainly through w3schools page:
www.w3schools.com/java/java_ref_reference.asp)

Specific references are located within code comments